尘客 · CHENKE

技术文档

给一群人推荐去哪玩 —— 意图控制、架构约束与可复现性

众口不再难调 · 只做深圳 · 202 个地点 · 125 个测试

项目名称：尘客（Chenke）

一句话：把「我们这群人去哪」拆成 16 个结构化约束，算出一组可执行方案，并说明谁让了步。

仓库：github.com/Chenk44/chenke　**版本：**v1.0.0（commit c7a14f5）

技术栈：Python 3.12 / FastAPI 0.162 / SQLAlchemy 2.1 / 原生 HTML+JS（零构建）

文档日期：2026-11-12　**页数：**不超过 30 页

本文档重点回答三个问题：① **AI 写代码时，我如何控制它想做的事（意图控制策略）；② 它写出来的代码被约束成什么架构；③ 别人拿到仓库能否跑出同样的结果（可复现性论证）。**

0 目录与阅读约定

- **1　项目与问题定位**
  - 1.1 它解决的到底是什么问题　1.2 规模与验收数字　1.3 本文档的证据来源
- **2　意图控制策略：怎么让 AI 做我要的东西**
  - 2.1 从一句话需求到可执行约束　2.2 上下文分五层　2.3 四类提示词骨架
  - 2.4 四类典型失控与对策　2.5 人机分工的边界　2.6 诊断优先于修复
  - 2.7 意图控制的账本
- **3　架构设计：AI 生成的代码被约束成了什么形状**
  - 3.1 四层与依赖方向　3.2 为什么算法层不认识框架　3.3 契约先行
  - 3.4 状态演化与迁移　3.5 降级链　3.6 复杂度预算　3.7 前端策略　3.8 约束怎么被强制
- **4　可复现性论证**
  - 4.1 结论先行　4.2 洁净检出实验　4.3 确定性的来源与残余风险
  - 4.4 外部依赖与断网策略　4.5 环境矩阵　4.6 数据与版本快照　4.7 复现失败时的定位手册
- **5　验收附录**
  - 5.1 测试分布　5.2 接口清单　5.3 数据字段与分布　5.4 已知局限（带数字）　5.5 三分钟复现清单

阅读约定

- **「我」**指项目作者（人类）；**「AI」**指编程助手。本文档描述的是两者之间的协作规程，不是工具测评。
- 文中所有数字都是**数出来或跑出来**的，不是估计：测试数来自 `pytest --collect-only`，行数来自逐文件 `splitlines()`，数据规模来自 `csv.DictReader`，提交数来自 `git rev-list --count`。
- 凡是我做错又改回来的地方，都保留在文中（第 2.4、3.4、4.7 节），因为它们比正确路径更能说明约束是怎么起作用的。
- 代码引用一律写成 `文件:行号` 或函数签名，可以直接在仓库里搜到。

1 项目与问题定位

1.1 它解决的到底是什么问题

真正的痛点不是「找不到好地方」，而是**一群人凑不出一个决定**。五个人的群聊里，每个人都能说出一个「不错的地方」，但没人愿意替别人拍板；讨论四十分钟，最后往往去了楼下商场。推荐系统解决的是「一个人去哪」，而这个问题里缺的东西恰恰是**公平**：不是平均分最高，而是别每次都让同一个人让步。

所以本项目的产品形态是三段：**建局**（拿到 6 位邀请码）→ **各填各的**（三个滑杆 + 硬条件 + 一句话）→ **输出三个方案 + 投票定案**。每个方案都同时给出「大家的共识度」和「谁让步最多」，把群体决策里最容易被忽视的信息摆到台面上。

> **范围是刻意收窄的：**只做深圳。一座城市的 202 个地点标得准，胜过十个城市各 20 条。这个决定同时约束了数据、算法与文档——后面所有「为什么不做得更大」的问题，答案都回到这一句。

1.2 规模与验收数字

202	20	125	17	39
深圳地点	商圈	自动化测试	HTTP 接口	提交（3 天）

指标	数值	口径 / 出处
地点数据	202 条 × 20 列	`data/pois_shenzhen.csv`，`csv.DictReader` 计数
商圈	20 个，每商圈 8–13 条	同上，`Counter(area)`
室内 / 室外	122 / 80	同上，按 `indoor` 列计
商圈通行表	190 对 × 2 = 380 行	`data/area_travel.csv`，高德路径规划预计算
代码规模	algo 1 319 / backend 2 068 / tests 1 291 / scripts 1 704 / web 806 行	逐文件 `splitlines()`，含空行与 `__init__.py`
测试	125 个（算法 40 / 接口 26 / 解析 29 / 营业时间 16 / 通行表 5 / 天气 9）	`pytest --collect-only -q`
提交	39 次，2026-11-10 10:01 → 11-12 19:20	`git rev-list --count HEAD`
单次推荐成本	1 005 次打分，约 3 毫秒	`docs/ALGORITHM.md` §8 实测

1.3 本文档的证据来源

三个问题分别对应三类证据，这也是本文档的组织方式：

- **意图控制** → 证据是**过程记录**：需求原话、我给出的选项、最终选择、因此产生的提交。
- **架构约束** → 证据是**代码事实**：依赖方向（`algo/` 里 grep 不到任何框架名）、模块职责、迁移函数、降级分支。
- **可复现性** → 证据是**实验输出**：把一个干净目录从 `git archive` 还原后，跑出来的每一条 stdout（第 4.2 节逐条贴出）。

2 意图控制策略：怎么让 AI 做我要的东西

这一节回答的第一个问题是：**AI 写代码时，「意图」到底在哪一层被决定？** 结论是：**不在提示词里，而在约束里**。提示词只能表达一次请求，约束才能让第二十次修改仍然朝着同一个方向。 所以我做的事可以概括成一句话——**把意图从「话」变成「可以被检查的东西」**。

2.1 从一句话需求到可执行约束

项目起点是一句很虚的话：「和朋友出去玩经常不知道去哪，需要一个软件综合各平台推荐，给出我们心目中的目的地」。 这句话里有三个陷阱：**「综合各平台」**（爬公开 API 拿不到的内容）、**「我们」**（多人，但没说怎么聚合）、**「心目中的」**（主观，无法验收）。 如果直接把这句话交给 AI，它会热情地开始写爬虫。

所以我先做的是**三次收敛**，每次都把一句模糊的话换成一个可检查的约束：

收敛	原始说法	换成可检查的约束	理由
数据源	「综合各平台推荐」	只用高德开放平台的 POI 结构化字段；9 个拿不到的字段自己人工标注	大众点评无公开 API、小红书反爬严格。**10 天里押爬虫，失败概率接近 1**；而且合规风险不可控
范围	「出去玩」（全国？哪都行？）	只做深圳，20 个商圈，每商圈 8–13 个点	一个人 10 天能标准的只有一座城市；范围一收，数据质量、算法参数、文档都能收敛
验收	「给出心目中的目的地」	输出 3 个「地点组合 + 时间线」，每个都带共识度与「谁让步最多」	「心目中的」不可验收，「共识度 0.73、最让步的是小林（0.71）」可验收

同一套做法也用在了**不做清单**上。明确写进文档「不做」的东西，AI 就不会在实现时顺手加回来： 实时聊天、社交关注、微信授权、小程序（审核 3–7 天等不起）、多城市、支付、后台管理。 （其中“不做小程序”这一条后来证明是对的：传智杯提交窗口只有十天，审核时间直接吃掉三分之一。）

2.2 上下文分五层

「上下文」这个词，在我的用法里不是指「给 AI 贴多少代码」，而是**五层逐级加严的约束载体**。 越往下越难违反，因为违反了会立刻报错：

层	载体	约束什么	违反了会怎样
L1	`README.md`（项目宪法）	做哪个城市、给谁用、砍掉什么、slogan、答辩口径	人读一遍就能发现跑题，改回来成本最低
L2	`docs/ALGORITHM.md`（公式）	每个系数的值、每个惩罚的方向、为什么这样定义公平	文档与代码不一致时，测试会红（测试注释直接引用章节号）
L3	`data/*.csv` + `algo/loader.py`	字段类型、枚举值、坐标范围、营业时间区间	启动即抛异常，不允许脏数据进内存
L4	`backend/app/schemas.py`	请求/响应形状、取值范围（如 `indoor_pref ∈ {-1,0,1}`）	422，并被压成人能看懂的文案
L5	`tests/` + CI	行为契约：同一输入必须给同一输出	CI 红，推不上去

五层里最有效的其实是 **L3 和 L5**，因为它们**不依赖我记性**。 举例：某次我让 AI 新增「品类偏好」字段，它很自然地想往 `CATEGORIES` 里塞一个新品类「书店」； 但 `algo/types.py` 里 7 个品类是封闭枚举， `loader.py` 会逐列校验， 于是它只能回头把「书店」映射到已有的「展览」—— **不是靠我提醒，是靠枚举挡住了**。

2.3 四类提示词骨架

我给 AI 的请求从来不是「帮我实现 X」，而是四种骨架之一。骨架的价值在于：**它把验收标准写进了请求本身**。

骨架	结构	真实例子（节选）	为什么有效
验收标准式	目标 + 可复现命令 + 期望 stdout + 不许破坏的既有行为	「把地点库扩到 150 条以上，要求：每个商圈至少 3 个室外点；`python -m scripts.seed` 要输出总条数；`pytest` 不许红；错标只改 CSV 一行就能修」	「至少 3 个室外点」是数字，能立刻检验；「错标改一行」决定了数据与代码必须解耦
诊断优先式	先分析、给选项、不要动代码；人拍板后再实现	「先不要急着修改代码，先分析问题出在哪然后告诉我」	避免「猜一个原因就打补丁」。本次诊断出**三层**原因，只改最上面一层会白干（详见 2.6）
降级要求式	主路径 +「XX 挂了也必须能跑」	「天气接口两个源都挂时，建局不能失败，退回手动选」	强制 AI 写出 `except` 分支，而不是让整条链路依赖外部服务
数字核对式	让 AI 自己数、自己改，并列出改了什么	「把 README 里的行数、测试数、地点数与代码对齐，逐条报差异」	文档里的数字是最容易腐烂的部分；本次核对揪出 9 处偏差（见 5.4）

> 「降级要求式」是我从失败里学到的。早期让 AI 接高德接口，它写得很干净——直到我拿真 Key 一跑， 发现「关键词搜出来的 typecode 不可信」：搜「剧本杀」首条返回的是咖啡厅的 typecode。**如果请求里没写「拿不到怎么办」，AI 默认假设「拿得到」。**

2.4 四类典型失控与对策

三天里 AI 反复犯的是同一批错误，所以我把对策也做成了固定动作：

失控类型	真实发生的一次	我的固定对策
悄悄改需求	让它「修好室内偏好」，它顺手把「必须室内」做成了硬过滤（直接删掉所有室外点）	要求任何「过滤」都必须说明**删掉了多少候选**；默认只降权不删除，除非全组一致（见 3.x 全员排除规则）
只加常量不改公式	为了「别让方案只有 3 小时」，它加了 `UNDERFILL_PENALTY = 0.15`，但忘了把它乘进 score，白跑一次	之后所有评分改动必须给出**同一输入改前/改后的方案对比**；没有对比就当没改
编数字	README 里出现过「1 062 行算法被 47 个测试覆盖」，而当时实际是 1 319 行 / 69 个测试	所有规模数字必须附一句**怎么数出来的**；改完代码后强制重数（数字核对式提示）
假装完成	「已修复」但真实数据库里旧列还在，一写入就 500（`preferences.require_indoor` NOT NULL）	验收标准里必须包含**在真实库、真实端口上跑一次**，不接受「单元测试过了」

2.5 人机分工的边界

我把决策分成两类，并且只对第一类负责：

- **不可逆 / 有代价的决策——人拍板。**做哪个城市、用什么技术栈、砍掉哪些功能、要不要接大模型、要不要公网。 这些决策一旦错了，三天时间补不回来，所以我要求 AI 每次只给 2–3 个选项 + 各自代价，由我选。
- **可逆 / 可验证的实现——AI 主导。**具体算法实现、SQL 迁移写法、CSS 排版、测试用例。 这类东西错了有编译器和 CI 兜着。

举个真实的分歧：我问「能不能把那一栏接入 AI 让它真听懂人话」。AI 的回答是先否掉一半——**「只接 AI 修不好这个 bug」**：因为问题有三层（词表没有、字段是布尔只能表达「要室内」、数据里某个商圈 5 个点全是室内）， 大模型只解第一层。然后它给了我 A/B/C 三个方案与各自工时，我选了 C（纯规则增强）。 这个选择后来被验证是对的：规则版解析 3 毫秒、可写测试、断网可用，而且真正让「不想去室内」生效的是**把布尔改成三态**，不是语言模型。

2.6 诊断优先于修复：一次完整的三层排查

用户报的原始问题是：「我说不想去室内，但依旧给我推荐室内去的地方，甚至所有人都说不想去但依旧推荐」。请求里我明确写了「先不要改代码」。AI 交回的诊断把问题拆成三层，每层都有可复现的证据：

层	证据	如果只修这一层会怎样
解析层	`nlp.py` 的室内词表只有「下雨 / 台风 / 太晒 / 高温」等天气词，**没有「室内」这个词本身**；实测 6 句话全部解析为「未提及」	补词表能听懂，但结果仍然一样——因为下一层不认
字段层	`Preference.require_indoor` 是布尔，只在为 `True` 时乘 0.2 惩罚；`False` 等于「没表态」	这是核心：**布尔表达不了「我不想」**，必须改成三态 `indoor_pref ∈ {-1,0,+1}` 并双向惩罚
数据层	49 个点里 26 个室内；而海岸城 5 个点**全部室内**，距离权重又是全表最高（0.30），「近的优先」必然导向室内	模型再对，池子里没有室外点也出不来室外方案

这次排查直接决定了后面三天的路线：**改词表与否定引擎 → 改字段为三态 → 把数据从 49 条扩到 202 条。** 如果当时直接按「加个 AI 就好」的思路动手，三层里只会修第一层，而这个 bug 至今还活着。

2.7 意图控制的账本

把三天的协作结果记成账，比任何描述都直观：

意图	落成的约束	可检查的结果
「不要玩具项目」	只做深圳 + 202 条人工标注 + 真实高德数据	数据体检脚本 `audit_pois.py` 报 0 处可疑
「不能是套壳开源项目」	自研效用函数 + 双聚合规则 + 组合搜索	69 个算法/解析测试钉住每个公式
「10 天 solo 能做完」	砍掉聊天/小程序/多城市/支付；技术栈按「能否独立写完 CRUD」定	3 天完成 39 次提交、125 个测试、CI 绿
「评委能跑起来」	零构建前端 + SQLite + 三条命令 + Dockerfile	洁净目录实测通过（第 4.2 节）

意图	落成的约束	可检查的结果
「别糊弄我」	数字核对式提示 + 已知局限带具体数字	自检揪出 9 处文档偏差并修正

3 架构设计：AI 生成的代码被约束成了什么形状

AI 写代码有一个稳定倾向：**把所有东西塞进路由函数里**，因为那样最短、最像「能跑」。 如果我只看结果不看结构，三天后会得到一个 800 行的 `main.py` ——能演示，但没法测、没法改、没法答辩。 所以架构不是「设计出来的」，是**用规则逼出来的**。这一节讲清四层结构、每层被允许做什么，以及这些规则靠什么强制。

3.1 四层与依赖方向

```
web/index.html           单页前端（零构建、零依赖，806 行）
      │  fetch /api/*
      ▼
backend/app/routers/     HTTP 层：校验入参、读写库、转格式（519 行）
      │
      ▼
backend/app/services/    翻译层 + 服务：nlp / weather / recommender（747 行）
      │
      ├──────────────→ algo/   纯算法包（1 319 行，不 import 任何框架）
      │
      ▼
backend/app/models.py + db.py   ORM 与迁移（303 行）
      │
      ▼
data/*.csv  ──seed──→  SQLite（chenke.db）
```

依赖方向是**单向**的，这条规则可以用一行命令检查：

```
$ Select-String -Path algo\*.py -Pattern 'fastapi|sqlalchemy|starlette|pydantic'
# （无输出）—— 算法层不认识任何一个 Web / ORM 库
```

3.2 为什么算法层不认识框架

这不是洁癖，是三个具体收益：

- **可测。**40 个算法测试直接 import `algo`，不需要数据库、不需要 HTTP、不需要起服务，跑完 3 秒。
- **可替换。**如果哪天要换 FastAPI → Flask，或者 SQLite → PostgreSQL，`algo/` 一行不用动。
- **可解释。**答辩时我能当场打开 `algo/utility.py` 指着公式说「这一项就是安静度的代价」，而不必在一堆 `@app.post` 里翻。

为了让这条边界在「AI 加功能」时不被冲垮，中间必须有一层**唯一翻译层**：`backend/app/services/recommender.py` 是**整个项目里唯一同时认识 ORM 和 `algo` 的文件**，它只做三件事——把 `POIRecord` 转成 `POI`、把 `PreferenceRecord` 转成 `Preference`、调用一次 `run_recommendation()`。换句话说：**想绕过算法层直接写业务逻辑，AI 得先绕过这个文件，而它只有 112 行。**

3.3 契约先行

「先定数据形状，再写逻辑」是我给 AI 的硬要求。项目里有三份契约：

契约	载体	校验方式	违反时的表现
数据契约	`data/pois_shenzhen.csv` 20 列	`algo/loader.py` 逐列校验：品类必须在 7 个枚举内、经纬度在深圳范围内、营业时间 `0 ≤ open < close ≤ 26`	启动即抛 `ValueError`，并指出是哪一行哪一列
接口契约	`backend/app/schemas.py`	Pydantic 模型 + 约束（`indoor_pref: ge=-1, le=1`）	422，并被全局 handler 压成一句人话
行为契约	`tests/` 125 个用例	同一输入必须同一输出（含时间线分钟数）	CI 红，推不上去

契约先行带来的一个意外好处：**字段可以先于数据库演化**。 解析器最早就能输出 16 个字段，但数据库只有 11 列； 于是 `merge_hints()` 写成「`if key in merged: 才写`」—— 解析器可以先上线、先显示灰色提示，等表结构补齐后再自动生效。这让「加字段」从一次性大改变成了三次小改。

3.4 状态演化与迁移

SQLite 的 `create_all()` 只会建表，**不会删列、不会改类型**。项目里踩过两次：

1. **补列**：`db.py` 里有一张 `_ADDED_COLUMNS` 表，启动时用 `PRAGMA table_info` 比对，缺哪列补哪列（`open_hour`、`start_hour`、`weather_text` 都是这样加进来的）。
2. **改语义**：把 `require_indoor`（布尔）改成 `indoor_pref`（三态）时，老库还留着 `require_indoor NOT NULL`， 新插入不带它就 500：`sqlite3.IntegrityError: NOT NULL constraint failed: preferences.require_indoor`。 修法是 `_drop_legacy_indoor_flag()`：改名老表 → 重建新表 → `INSERT ... SELECT ..., CASE WHEN require_indoor THEN 1 ELSE 0 END` → 删老表。

> **这里有一次真实的失败，值得记下来：**第一次跑迁移报 `index ix_preferences_member_id already exists` —— 因为 **SQLite 的索引名是全库唯一的**，把表改名后旧索引还在，新建表想用同名索引就撞了。 正确顺序是：**先收集旧索引名 → 改名 → `DROP INDEX` → 建表 → 拷数据 → 删老表**。 修好之后我造了一个老 schema 的库（含 `require_indoor`、唯一索引、两行数据）做回归，输出确认列已变成 `indoor_pref`、数据行数不变。

3.5 降级链

这个项目有四个外部依赖，每一个都可能挂。架构上要求：**任何一个挂了，页面仍然能用**，只是精度下降。

依赖	失效表现	降级动作	用户看到什么
高德路网（商圈通行）	表里查不到这对商圈 / 没网	退回球面距离 + 地铁/驾车两套参数估算	方案照常出，只是时间略粗；误差约 20%（最大例子：海岸城→龙华估算 40、高德公交 83 分钟）

依赖	失效表现	降级动作	用户看到什么
高德天气（4 天）	接口超时	换 Open-Meteo（7 天）；两个都挂 → 只影响「能选几天」	建局不失败，日期选择器仍然可用，点查不到时提示「可以先手动选天气」
两个天气源都挂	预报整表为空	返回 `in_range=true, kind=null`	页面显示「这天的预报暂时没查到」，不拦建局
邀请码生成	重名冲突	6 位码循环重试；成员重名按昵称幂等	无感

降级链的设计原则是**「坏消息要说清楚，但不许拦路」**：能算就算，算不准就标注，绝不因为一个可选数据源不可用就把主流程打断。

3.6 复杂度预算

AI 很容易把「优化」当成「上算法」。这个项目的搜索空间其实很小，所以架构上明确写死：**枚举，不上遗传算法**。

阶段	规模	实测
单点打分	202 个地点 × 5 人	1 005 次打分
候选池	共识分前 24（最多 +4 个餐饮兜底）	—
组合枚举	C(20,3) = 1 140 种组合 × 6 种顺序	约 3 毫秒
多样性去重	Jaccard 阈值 0.20 → 0.34 → 1.01 逐级放宽	保证三个方案不共用前两个点

「这个规模枚举就够了」在答辩里是个加分项：它说明我知道**什么时候不该上复杂算法**， 而不是把遗传算法当勋章。同理，第 3.7 节的前端也不用框架——单文件 806 行，评委双击就能开。

3.7 前端策略

前端只做了一个决策：**零构建**。一个 `web/index.html`，内联 CSS 与 JS，不装 npm、不打包、不依赖 CDN。 代价是我得手写状态管理（一个 `state` 对象 + `localStorage` 两个键）；收益是：

- 评委 clone 下来 **只需要 Python**，不必先跑 `npm install`（这一步在国内网络下经常卡十分钟）；
- 静态页由 FastAPI 直接挂载，**改前端不用重启服务**（每次请求读磁盘）；
- 整个产品只有 64 个被跟踪文件，最大的是两张截图（161 KB）。

3.8 约束怎么被强制

诚实地说：**分层依赖没有被工具自动检查**（Python 没有类似 Java 的模块依赖校验）。 它能保持，靠的是三件半自动的事：

1. **CI 的四道门**：`ruff check` → `python -m scripts.seed`（数据契约）→ `pytest`（行为契约）→ 真起服务 `curl /api/health` 与 `/api/areas`（端到端）。
2. **一行 grep**：新增 `algo/` 文件时，跑一次第 3.1 节那条命令，有输出就说明层被污染了。

3. **文件大小直觉**：`recommender.py` 112 行、`routers/` 每个文件不到 350 行。某个文件突然变两倍，通常意味着职责混了。
4. **文档里写明禁令**：`docs/ALGORITHM.md` 开头就写着「本文档对应 `algo/` 里的每一个函数」，AI 每次改算法都要顺手改这一节，改不动就说明改错了层。

4 可复现性论证

4.1 结论先行

任何一台装了 Python 3.10+ 的机器，clone 仓库后执行三条命令，就能得到本文档描述的同一套结果：

```
pip install -r requirements.txt
python -m scripts.seed                          → 期望输出：共 202 条
python -m uvicorn app.main:app --app-dir backend   → 打开 http://127.0.0.1:8000
```

「同一套结果」指四件事：**202 条数据进库、125 个测试全绿、示例局的推荐结果与文档一致、页面能打开并投票**。 下面用一次真实实验来证明，而不是断言。

4.2 洁净检出实验

实验方法：**不用现有工作目录**，而是把 `HEAD` 导出成一个纯文本归档，解到一个空目录里跑。 这样能同时验证「仓库里该有的东西都有」和「不该有的东西（虚拟环境、数据库、密钥）都没被误提交」。

#	命令	实际输出（原文）
0	`git archive --format=tar HEAD -o clean.tar` → 解到空目录	还原出 18 个顶层条目；`.venv` / `.env` / `*.db` / `.tmp` **均不存在**
1	`python -m scripts.seed`	✅ 地点库已更新：新增 202 条，更新 0 条，共 202 条
2	`pytest -q tests`	`125 passed, 1 warning in 11.88s`
3	`python -m scripts.demo`	exit code `0`；终端打印 3 个方案的共识度与投票结果
4	`uvicorn app.main:app --port 8021`	`/api/health` → `{"status":"ok","app":"尘客","version":"0.1.0"}`
5	`GET /api/areas` / `GET /api/pois`	20 个商圈；202 个地点
6	`POST /api/demo` → `POST /api/groups/{code}/recommend`	3 个方案，共识度 `0.735 / 0.691 / 0.645`，notes 为 `["💧检测到降雨，已自动优先室内地点"]`
7	`GET /`	首页 33 406 字节（单页应用本体）

注：实验在 Windows 10 + Python 3.12.14 上进行；端口换 8021 是为了不干扰已在 8000 运行的实例。 CI 上等价的一组动作（Linux + Python 3.10/3.12）由 `.github/workflows/ci.yml` 每次 push 自动执行，当前为绿。

4.3 确定性的来源与残余风险

「可复现」不只是「能跑」，还要「跑出同样的东西」。项目里可能引入不确定性的地方，我逐个查过：

不确定源	是否存在	处理
随机数	**算法层没有**	全仓库唯一用到 `random` 的地方是 `routers/groups.py` 生成 6 位邀请码——它是标识，不参与排序
集合/字典遍历顺序	不影响结果	集合只用于去重与成员判断，**所有对外输出都先排序**；排序键是浮点分数，同分时靠 Python 稳定排序保持枚举顺序
时间 / 时区	影响天气与时间线	时间线由 `start_minute`（整数分钟）+ 偏移量决定，与系统时钟无关；天气按 `Asia/Shanghai` 取日期，缓存 30 分钟
浮点累加	存在	对外只输出 1–2 位小数；测试断言用 `pytest.approx` 而不是等值
网络	不影响测试	9 个天气测试全部 `monkeypatch` 掉 `forecast_map`，CI 无网也全绿

4.4 外部依赖与断网策略

这个项目在最坏情况下（无网、无 Key、无云）依然能完成「建局 → 填偏好 → 出方案 → 投票」的完整闭环。原因是两个依赖都被**提前固化进了仓库**：

- **通行时间**：190 对商圈 × 驾车/公交 = 380 行，已经用高德路径规划算好写进 `data/area_travel.csv`（11 KB），跟仓库一起走。 不预计算的代价是：一次推荐要算 24^2 量级的两两时间，免费配额会被枚举打光。
- **天气**：只有「选日期自动查天气」用到网络；查不到就退回手动选，**建局不会失败**。
- **数据标注**：9 个人工字段是离线完成并写进 CSV 的，运行时一个外部请求都不发。

4.5 环境矩阵

环境	状态	备注
Windows 10 + Python 3.12.14	开发环境，全绿	125 测试通过；本机服务 /api/health 200
Linux + Python 3.12（GitHub Actions）	绿	每次 push 自动跑：ruff → seed → pytest → 起服务 curl
Linux + Python 3.10（GitHub Actions）	绿	验证没有用到 3.11+ 语法
Docker（Dockerfile 多阶段）	已提供未本地验证	仓库有 `Dockerfile`（32 行，非 root，`CMD seed && uvicorn`），作者本机未安装 Docker，此项属于**未验证项**，如实说明

4.6 数据与版本快照

为了让「同一套数据」可被核对，文档里给出关键文件的 SHA-256 前缀（完整值可用 `certutil -hashfile` 或 `sha256sum` 复核）：

文件	大小	SHA-256（前 16 位）
`data/pois_shenzhen.csv`	36 312 B	`cba9cfb9f5f3b9c6`
`data/area_travel.csv`	11 549 B	`e68d7c15706d8d7c`
`requirements.txt`	430 B	`9211f0ecfcf08fdc`

文件	大小	SHA-256（前 16 位）
`web/index.html`	36 401 B	`afc369798c9f9c7c`

开发环境实测版本（不是「大于等于」，是当时真实装上的）：`fastapi 0.162.2`、`starlette 1.10.0`、`pydantic 2.14.3`、`SQLAlchemy 2.1.8`、`uvicorn 0.54.0`、`pytest 9.2.0`、`httpx 0.28.1`。运行时依赖只有 7 个（`requirements.txt` 里可数）。

4.7 复现失败时的定位手册

按概率从高到低列出我实际踩过的坑。这些不是理论风险，全部真实发生过并已修：

现象	根因	修法（仓库里已有的）
`ModuleNotFoundError: app`	没加 `--app-dir backend`	按 README 的三条命令，或直接用 `run.ps1` / `run.sh`
`sqlite3.IntegrityError: NOT NULL constraint failed`	用老版本数据库文件跑新代码	删掉 `chenke.db` 重新 `seed`；或让 `init_db()` 自动迁移（已实现）
中文在 Windows 控制台乱码 / 脚本报「字符串缺少终止符」	PowerShell 5.1 按 ANSI 读无 BOM 的 UTF-8 脚本	`run.ps1` 已存成 **UTF-8 with BOM**；`seed.py` 里也做了 `sys.stdout.reconfigure(encoding="utf-8")`
改了代码但接口行为没变	访问 `127.0.0.1` 命中了另一个更早启动的进程（Windows 允许 `0.0.0.0:8000` 与 `127.0.0.1:8000` 同时监听）	`netstat -ano | findstr :8000` 查占用，只保留一个；日常用 `--reload`
`git push` 超时但网页能开	DNS 把 `github.com` 解析到不可达的 IP	与代码无关；换网络或配代理后重试
天气接口不通导致建局失败	早期版本未实现降级	已实现：查不到退回手动选，**建局永不失败**

5 验收附录

5.1 测试分布

文件	用例数	钉住的是什么
`tests/test_algo.py`	40	效用函数的每一项、共识分、Borda、minimax regret、组合搜索、时间线偏移、加载器校验、全员排除、三态室内偏好
`tests/test_api.py`	26	17 个接口的端到端行为、重名幂等、投票并列、422 文案、首页必须给「回首页」的出口
`tests/test_nlp.py`	29	一句话解析：否定、条件句、品类、时长、出发时间、与滑杆的优先级
`tests/test_hours.py`	16	高德时间串解析、跨零点（18:00–02:00）、打烊降权、老 CSV 无列时的默认

文件	用例数	钉住的是什么
`tests/test_travel.py`	5	通行表查询、驾车/公交不同、查不到就退估算、表必须随仓库发布
`tests/test_weather.py`	9	天气词到四种类别的映射、7 天窗口、超范围提示、手动优先于预报、**无预报仍能建局**

5.2 接口清单

共 17 个，全部有真实样例（`docs/API.md`），并自动生成 OpenAPI 文档（`/docs`）：

方法	路径	作用
GET	`/api/health`	健康检查（容器探针 / CI 用）
POST	`/api/demo`	一键生成填好的示例局（5 人，进来就能看结果）
POST	`/api/groups`	建局，返回 6 位邀请码
GET	`/api/groups/{code}`	局的信息 + 成员列表
POST	`/api/groups/{code}/members`	用昵称入局（重名幂等）
PUT	`/api/members/{id}/preference`	填/改偏好，可带一句话自动解析
GET	`/api/members/{id}/preference`	读偏好
POST	`/api/parse`	只解析一句话、不保存（前端实时提示）
POST	`/api/groups/{code}/recommend`	生成 3 个方案
GET	`/api/groups/{code}/plans`	读取上次生成的方案
POST	`/api/plans/{id}/vote`	投票（幂等）
GET	`/api/groups/{code}/votes`	票数统计（并列都标最高）
GET	`/api/pois`	地点列表，支持商圈/品类/室内过滤
GET	`/api/areas`	商圈列表
POST	`/api/reviews`	玩完之后的评价
GET	`/api/pois/{id}/reviews`	某个地点的评价
GET	`/api/weather?date=YYYY-MM-DD`	查那天的天气（7 天内；查不到会说查不到）

5.3 数据字段与分布

20 列里，**11 列来自高德**（名称、坐标、行政区、地址、电话、评分、人均、营业时间、商圈距离等），**9 列是自己标注的**（商圈归属、室内外、安静度、人数上下限、带娃、预约、排队、标签、一句话理由）。 没有爬任何点评类站点——这是合规底线，也是答辩时第一个可能被问到的点。

品类	条数	品类	条数
公园	58	餐饮	22
玩乐	38	咖啡	18

品类	条数	品类	条数
展览	34	夜生活	10
影院	22	合计	**202**

室内 122 / 室外 80，**每个商圈至少 3 个室外点**——这条数字是硬要求，因为它是「说不想去室内却推室内」那个 bug 的数据层根因。

5.4 已知局限（带数字）

这一节不是免责声明，是**取舍清单**。能修的四条已经修完（营业时间、通行时间、全员排除、数据体检），剩下的三条是刻意不做：

1. **只做了深圳。**换城市要重跑 `fetch_amap → shortlist → draft_labels → backfill_hours`，再人工复核约 2 天。
2. **一句话解析是规则版，不是大模型。**16 个字段、约 3 毫秒、可写测试、断网可用；代价是听不懂太绕的说法。 这是刻意的：让模型只做「人话 → 结构化约束」，推荐结果仍由可解释的效用函数决定。
3. **营业时间只到「这个点还开不开」，不到「今天开不开」。**没有节假日数据，元旦去美术馆可能吃闭门羹。

另外两条是「做了但没做到底」，也写清数字：通行时间表只覆盖 20 个商圈两两（190 对），同商圈内部仍是估算，误差约 20%； 202 条里有一半是脚本先猜、人工复核，`scripts/audit_pois.py` 目前报 **0 处可疑**，发现错标改一行 CSV 再 `seed` 即可。

5.5 三分钟复现清单

1. `git clone https://github.com/Chenk44/chenke && cd chenke`
2. `python -m venv .venv` → 激活 → `pip install -r requirements.txt`（7 个依赖，无编译）
3. `python -m scripts.seed` → 看到 **共 202 条**
4. `python -m uvicorn app.main:app --app-dir backend`
5. 浏览器打开 `http://127.0.0.1:8000`，点「先看一个示例局」→ 五秒后看到三个方案
6. （可选）`pytest -q` → 看到 **125 passed**；`python -m scripts.demo` → 终端里跑完整流程

6 附录：需求原话与它落成的约束

这一节把第 2 节的方法落到证据上：下面是三天里**原话**（未改写，只去掉语气词），以及我当时从这句话里提取出的、**可以检查的约束**。它同时也是「我不是想到哪做到哪」的证明——每一次需求变更都能对应到一次提交或一条测试。

需求原话	我提取的约束	落在哪
「开发一款应用…综合各平台推荐…给出我们心目中的目的地」	数据源只能用合法结构化数据；输出必须说清「谁让了步」	§2.1、README 第 1 节
「python吧」	技术栈锁定 Python；动态类型的短板用 pytest + ruff + 类型注解 + CI 补回来	`pyproject.toml` 、 `.github/workflows/ci.yml`
「先做C。纯规则增强…我想把能去玩的地点再增加，你先帮我列出来」	先给清单让人选，不许直接开工；规则侧目标 16 字段；数据 49 → 202，每商圈至少 3 个室外点	§2.3 验收标准式； `data/pois_shenzhen.csv`
「with_elder没必要，你先验证一下高德api吧」	砍功能要明确记进「不做清单」；外部依赖先拿真 Key 探活再写代码	§2.1 不做清单； `scripts/fetch_amap.py`
「先不要急着修改代码，先分析问题出在哪然后告诉我」	诊断先于修复；结论必须分层，每层给可复现证据	§2.6 三层排查表
「1.生成完一轮之后我无法退回主页面　2.我说不想去室内，但依旧给我推荐室内」	两个独立缺陷；第二个拆成「听不懂 / 字段表达不了 / 数据里没有」三层	§2.6、 `indoor_pref` 三态、 `test_page_always_offers_a_way_back_home`
「1」（回选项）	凡是有多条路线，先给 A/B/C 和各自代价，只回编号就能推进	§2.3 选项式提问
「我想直接选择日期然后调用那一天的天气（一周之内的天气）」	窗口 7 天；查不到要说「查不到」而不是猜；接口挂了建局也不能失败	§3.5 降级链、 `services/weather.py`
「不需要公网了，没这个需求」	删掉隧道脚本与文档段落，不保留「以后可能有用」的死代码	提交 `412805c`
「首先先把公网相关的可以删了，然后自检吧」	自检必须是数字核对，不是「我检查过了」	§2.4 数字核对式、§4.6 快照表

6.1 文中每个数字的核对命令

评委如果想验证这份文档没在编数字，下面四条命令可以复现文中绝大多数统计。输出应与 §1.2 的规模表、§4.6 的快照表逐位一致；不一致说明文档腐坏了，按 §2.4 的「数字核对式」重新生成，而不是手工改一个数字了事。

```
# 测试数
python -m pytest --collect-only -q tests | tail -1

# 代码行数（含空行、含 __init__.py）
python -c "import pathlib;[print(f, len(pathlib.Path(f).read_text(encoding='utf-
8').splitlines())) for f in ['algo/utility.py']]"

# 数据规模
python -c "import
csv;print(len(list(csv.DictReader(open('data/pois_shenzhen.csv',encoding='utf-8')))))"

# 提交数与文件指纹
git rev-list --count HEAD
python -c "import
hashlib;print(hashlib.sha256(open('web/index.html','rb').read()).hexdigest()[:16])"
```

6.2 被拒绝的三条技术路线

同样重要的是「没做什么」。下面三条我都认真算过成本，也都在不同时间点上放弃了， 理由写在这里，因为答辩时它们比成功路径更容易被问到：

路线	如果做了会怎样	为什么放弃
让大模型直接推荐地点	十行代码就能出「听起来很美」的答案，演示效果立刻变好	① 评委问「为什么是这家」时只能回答「模型说的」；② 断电断网即废； ③ 同样的输入不保证同样的输出， §4.3 的确定性论证整节作废
爬大众点评 / 小红书补「多平台」	数据量能翻十倍，也更像「综合各平台」	① 都没有公开 API，反爬强度高；② 10 天窗口里失败概率接近 1； ③ 合规风险不可控——这一条足以让作品在合规审查上直接出局
做微信小程序 / 接微信登录	分享到群里更顺手，看起来更「完成」	① 小程序审核 3–7 天，评审窗口只剩 10 天；② 授权要企业主体； ③ 同一套 H5 + 6 位邀请码已经能覆盖「发到群里大家各填各的」这个核心动作

这三条的共同点是：**它们都能让演示更好看，但都会削弱「这东西为什么可信」**。 在一个三天能跑起来的项目里，删掉一条路线省下的不是代码量，而是答辩时无法回答的问题。

— 文档结束 —